

Neuronales Netzwerk von Grund auf

Formeln und Intuition

Daniel Stein

1 Grundidee

Ein neuronales Netzwerk besteht aus mehreren Schichten (Layern). Jede Schicht transformiert Eingabewerte in neue Werte.

Ziel ist es, eine Funktion $f(x)$ zu lernen, die Eingaben x auf gewünschte Ausgaben y abbildet.

2 Notation und Bedeutung

- $a^{(l)}$ = Aktivierungen (Ausgabe der Schicht l)
- $z^{(l)}$ = gewichtete Summe vor Aktivierung
- $W^{(l)}$ = Gewichtsmatrix
- $b^{(l)}$ = Bias-Vektor
- $\delta^{(l)}$ = Fehler (wie stark beeinflusst ein Neuron den Loss)

Wichtig:

- Die Gewichtsmatrix $W^{(l)}$ verbindet Layer $l - 1$ mit Layer l
- $W_{ij}^{(l)}$ sagt: wie stark beeinflusst Neuron j aus Layer $l - 1$ das Neuron i in Layer l

Input:

$$a^{(0)} = x$$

3 Forward Pass

3.1 Intuition

Jedes Neuron:

- sammelt Eingaben
 - gewichtet sie
 - addiert einen Bias
 - wendet eine Aktivierungsfunktion an
-

3.2 Formeln

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = f^{(l)}(z^{(l)})$$

Ein einzelnes Neuron:

$$z_i^{(l)} = \sum_{j=1}^{n_{l-1}} W_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$$

4 Aktivierungsfunktionen

Sie bringen Nichtlinearität ins Netzwerk.

4.1 Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

4.2 ReLU

$$\text{ReLU}(z) = \max(0, z)$$

$$\text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

4.3 Softmax

Wird für Klassifikation verwendet:

$$a_i^{(L)} = \frac{e^{z_i^{(L)}}}{\sum_j e^{z_j^{(L)}}}$$

Ergebnis ist eine Wahrscheinlichkeitsverteilung.

5 Loss-Funktion

Misst den Fehler zwischen Vorhersage und Ziel.

5.1 Mean Squared Error

$$L = \frac{1}{2} \sum_i (a_i^{(L)} - y_i)^2$$

5.2 Cross-Entropy

$$L = - \sum_i y_i \log(a_i^{(L)})$$

6 Backpropagation (Kernidee)

Ziel: herausfinden, wie jede Gewichtung den Fehler beeinflusst.
Grundidee:

- Fehler wird vom Output zurück propagiert
 - jeder Layer bekommt einen Anteil am Fehler
-

6.1 Definition von δ

$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}$$

Interpretation: Wie stark beeinflusst $z^{(l)}$ den Gesamtfehler?

6.2 Output Layer

MSE:

$$\delta^{(L)} = (a^{(L)} - y) \odot f'(z^{(L)})$$

Softmax + Cross-Entropy:

$$\delta^{(L)} = a^{(L)} - y$$

6.3 Hidden Layer

$$\delta^{(l)} = \left((W^{(l+1)})^T \delta^{(l+1)} \right) \odot f'(z^{(l)})$$

Interpretation:

- Fehler wird zurück verteilt
 - und lokal angepasst durch die Ableitung
-

7 Gradienten

7.1 Gewichte

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T$$

Einzelnes Gewicht:

$$\frac{\partial L}{\partial W_{ij}^{(l)}} = \delta_i^{(l)} \cdot a_j^{(l-1)}$$

Interpretation:

- großes δ = großer Fehler
- großes a = starker Einfluss

—

7.2 Bias

$$\frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}$$

—

8 Gradient Descent

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}$$

$$b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}$$

η ist die Lernrate.

—

9 Batch Training

Für mehrere Trainingsdaten:

$$\frac{\partial L}{\partial W} = \frac{1}{N} \sum_{k=1}^N \nabla_W L^{(k)}$$

—

10 Gesamtablauf

1. Input setzen: $a^{(0)} = x$
2. Forward Pass
3. Loss berechnen
4. Backpropagation
5. Gewichte aktualisieren

—

11 Wichtigste Formel

$$\frac{\partial L}{\partial W_{ij}^{(l)}} = \delta_i^{(l)} \cdot a_j^{(l-1)}$$